DEPI SOFTWARE TESTING TRACK — FINAL PROJECT

Risk Assessment & Mitigation Plan

Risk Governance Lifecycle, Scoring Matrix, Heat Maps, 24 Detailed Risk Cards, Contingency Scenarios, and Descoping Order

Project	Banking Transaction System	**Document**	Risk Assessment & Mitigation Plan
Prepared By	Muhammad Sayyid (Tech Lead)	**Reviewed By**	Project Team

Part 1. Purpose & Scope

Proactive identification, scoring, prevention, and contingency planning for technical, schedule, security, and operational risks.

Scope & Importance: This document identifies potential technical, operational, security, and schedule risks in the Banking Transaction System project, quantifies their severity, specifies preventive mitigation actions, and establishes actionable contingency plans (Plan B) if a risk materializes. It also assigns clear ownership and monitoring frequency for every risk across the entire 8-week project lifecycle (planning, full-stack development, automated and manual testing, security auditing, documentation, and final live defense).

Because the project is executed by a five-member team over an 8-week schedule using multiple frameworks (Spring Boot 3.5.x, SQLite, Thymeleaf, JUnit 5, Selenium 4, and Postman) to build a financial system where precision and security are paramount, a written risk plan enables the team to prevent defects early rather than reacting late.

Part 2. Risk Management Approach

Structured six-step risk lifecycle and role-based governance responsibilities.

2.1 Six-Step Risk Management Process

Step #	Lifecycle Stage	Team Activity & Execution Protocol	Expected Output
1	Identify	Collect risks from requirement reviews, architecture design, initial SQLite/Spring spikes, and weekly team syncs. Any team member can raise a risk.	New risk entry in the Risk Register
2	Analyse	The assigned Risk Owner evaluates Likelihood (1–5) and Impact (1–5) using the standardized scoring scales in Part 3.	Inherent Risk Score (1–25) and Rating Band
3	Evaluate	Rank risks by score to determine priority, required governance attention, and response urgency.	Prioritized Risk Register & Top 10 List
4	Respond	Select a response strategy (Avoid, Mitigate, Transfer, Accept), define preventive actions, set early-warning indicators, and document the Plan B contingency.	Detailed Risk Card with Residual Score

Step #	Lifecycle Stage	Team Activity & Execution Protocol	Expected Output
5	Monitor	Review top risks weekly (15-minute slot in the team meeting); update scores, status, and early-warning signs in Jira and the register.	Updated Risk Status & Jira Action Items
6	Close	Close a risk when the triggering event can no longer occur or when its residual rating remains Low for three consecutive weekly reviews.	Closed Risk Record

2.2 Risk Governance Roles & Responsibilities

Role	Assigned Person(s)	Risk Management Responsibilities
Risk Plan Lead & Architect	Muhammad Sayyid (Tech Lead)	Authors and maintains the Risk Assessment & Mitigation Plan; owns architecture, database concurrency, security, and Git workflow risks; enforces branch/PR quality gates.
Risk Coordinator & QA Reviewer	Somia Rashad (QA Lead)	Chairs the weekly 15-minute risk review, tracks testing/requirement risks and Jira mitigation items, and verifies that every risk has a named owner and target date.
Risk Owners	Assigned Role per Risk Card	Executes preventive mitigation actions, monitors early-warning indicators continuously, triggers the contingency plan when needed, and updates risk scores.
Project Team	All 5 Team Members	Approves responses for High and Critical risks, agrees on scope cuts (descoping order), reports warning signs immediately in team chat, and follows mitigation rules.
Track Instructor	DEPI Software Testing Instructor	Informed promptly regarding any Critical risk or any issue that threatens a core milestone deliverable or the final presentation schedule.

Part 3. Risk Scoring Methodology

Each risk is scored on Likelihood (1 to 5) and Impact (1 to 5). The Risk Score equals Likelihood × Impact (range 1 to 25). Inherent Score measures risk before mitigation; Residual Score measures expected risk after mitigation controls are active.

3.1 Likelihood Scale (1 to 5)

Score	Level	Probability Range	Practical Meaning in This Project
1	Rare	Below 10%	Requires a highly unusual or external event to occur.
2	Unlikely	10% to 30%	Could happen theoretically, but there are currently no warning signs.
3	Possible	30% to 50%	Has occurred in similar 8-week academic or capstone software projects.

Score	Level	Probability Range	Practical Meaning in This Project
4	Likely	50% to 80%	Expected to happen during development or testing unless the team takes preventive action.
5	Almost Certain	Above 80%	Will almost certainly happen without immediate structural intervention.

3.2 Impact Scale (1 to 5)

Score	Level	Practical Impact on Schedule, Scope, Quality, or Integrity
1	Negligible	No noticeable effect on the 8-week schedule, deliverables, or test results.
2	Minor	Up to 1 day of delay, or a minor cosmetic UI issue with an easy workaround.
3	Moderate	Approximately 1 week of delay, loss of a 'Should-have' feature, or weaker testing evidence.
4	Major	A core 'Must-have' feature is missing or unstable, a required deliverable is incomplete, or a security / money-integrity bug remains.
5	Severe	The application cannot be demonstrated live, or financial balances/transactions cannot be trusted at all.

3.3 Rating Bands & Required Response Governance

Rating Band	Score Range (L × I)	Mandatory Governance & Escalation Response
Critical	**17 to 25**	Act immediately. Escalate the same day to the entire team. Consult the Track Instructor if a deliverable is threatened. Freeze new feature work in the affected module until a remediation plan is active.
High	**10 to 16**	Mandatory preventive mitigation actions with a named owner and target date. Reviewed weekly in the team meeting. Whole team approves the mitigation plan.
Medium	**5 to 9**	Implement cost-effective preventive mitigations. Reviewed weekly. Assigned Risk Owner monitors early-warning indicators.
Low	**1 to 4**	Accept and monitor. Re-evaluated at major milestone checkpoints (Weeks 1, 4, 6, and 8).

Part 4. Risk Response Strategies

Four standard risk response strategies applied across the Banking Transaction System.

Strategy	Definition	Concrete Application in the Banking Transaction System
Avoid	Modify the architecture or project plan so the risk cannot occur.	Run the final demonstration locally with embedded SQLite rather than deploying to a public cloud server, avoiding hosting outages and internet exposure risks.

Strategy	Definition	Concrete Application in the Banking Transaction System
Mitigate	Take proactive technical or process actions to reduce Likelihood, Impact, or both.	Store all monetary values as integer whole piasters (BIGINT) in SQLite with WAL mode and execute parallel withdrawal concurrency tests (TC-C-01) to prevent money-integrity bugs.
Transfer	Shift the technical burden of risk handling to a proven framework or library.	Delegate password hashing (BCrypt), CSRF token synchronization, and role filter chains to Spring Security rather than writing custom cryptography or session management.
Accept	Acknowledge a low-impact constraint, take no code action now, and document it.	Accept that Spring Boot 3.5.x ended free commercial updates in June 2026 for this local academic demo, verify dependencies via OWASP scanner, and document the Spring Boot 4.1 upgrade path.

Part 5. Executive Risk Summary & Heat Maps

The Risk Register contains 24 identified risks. Prior to mitigation, there are 13 High risks, 9 Medium risks, 2 Low risks, and 0 Critical risks. After applying the defined mitigation controls, High risks drop to 0 (13 Medium and 11 Low).

5.1 Risk Count by Rating Band (Inherent vs. Residual)

Assessment Stage	Critical (17–25)	High (10–16)	Medium (5–9)	Low (1–4)	Total Risks
Inherent Risk (Before Mitigation)	0	13	9	2	24
Residual Risk (After Mitigation)	0	0	13	11	24

5.2 Heat Map Before Mitigation (Inherent Risk Distribution)

Likelihood \ Impact	1: Negligible	2: Minor	3: Moderate	4: Major	5: Severe
5: Almost Certain	—	—	—	—	—
4: Likely	—	—	**R-07, R-08, R-15, R-21 (Score 12)**	**R-01, R-02, R-17 (Score 16)**	—
3: Possible	—	**R-10 (Score 6)**	**R-04, R-09, R-12, R-16, R-18 (Score 9)**	**R-06, R-11, R-20, R-24 (Score 12)**	**R-03, R-19 (Score 15)**
2: Unlikely	—	**R-22, R-23 (Score 4)**	—	**R-05, R-13, R-14 (Score 8)**	—
1: Rare	—	—	—	—	—

5.3 Heat Map After Mitigation (Residual Risk Distribution)

Likelihood \ Impact	1: Negligible	2: Minor	3: Moderate	4: Major	5: Severe
5: Almost Certain	—	—	—	—	—

Likelihood \ Impact	1: Negligible	2: Minor	3: Moderate	4: Major	5: Severe
4: Likely	—	—	—	—	—
3: Possible	—	**R-21 (Score 6)**	**R-01, R-02, R-17 (Score 9)**	—	—
2: Unlikely	—	**R-09, R-10, R-12, R-15, R-18 (Score 4)**	**R-04, R-07, R-08, R-11, R-16, R-19, R-24 (Score 6)**	**R-03, R-06 (Score 8)**	—
1: Rare	—	**R-22, R-23 (Score 2)**	**R-13, R-14, R-20 (Score 3)**	**R-05 (Score 4)**	—

5.4 Top 10 Project Risks (Ranked by Inherent Score)

ID	Risk Title	Inherent Score	Inherent Rating	Risk Owner	Residual Score & Rating
R-01	Scope too large for the available 8-week schedule	16 (4×4)	**High**	Tech Lead (Muhammad Sayyid)	**9 — Medium (3×3)**
R-02	Skill gap across team members on new frameworks/tools	16 (4×4)	**High**	Tech Lead (Muhammad Sayyid)	**9 — Medium (3×3)**
R-17	Limited member availability due to university exams and courses	16 (4×4)	**High**	Whole Team	**9 — Medium (3×3)**
R-03	Dependence on one or two members for reviews and core architecture	15 (3×5)	**High**	Tech Lead (Muhammad Sayyid)	**8 — Medium (2×4)**
R-19	Live demonstration environment or database failure on defense day	15 (3×5)	**High**	Tech Lead (Muhammad Sayyid)	**6 — Medium (2×3)**
R-06	Money-integrity calculation errors or race-condition concurrency bugs	12 (3×4)	**High**	Tech Lead (Muhammad Sayyid)	**8 — Medium (2×4)**
R-07	Ambiguous, contradictory, or changing business requirements	12 (4×3)	**High**	QA Lead (Somia Rashad)	**6 — Medium (2×3)**
R-08	Flaky or fragile Selenium WebDriver UI automation tests	12 (4×3)	**High**	QA Lead (Somia Rashad)	**6 — Medium (2×3)**
R-11	Application security weaknesses (IDOR, broken access control, XSS, CSRF)	12 (3×4)	**High**	Tech Lead (Muhammad Sayyid)	**6 — Medium (2×3)**
R-15	Git merge conflicts and broken builds on the main branch	12 (4×3)	**High**	Tech Lead (Muhammad Sayyid)	**4 — Low (2×2)**

Part 6. Complete Risk Register (R-01 to R-24)

Master index of all 24 project risks showing Likelihood (L), Impact (I), Inherent Score/Rating, Owner, and Residual Score/Rating.

ID	Category	Risk Title	L	I	Inherent	Owner	Residual
R-01	Schedule & Scope	Scope too large for the available 8-week time	4	4	**16 (High)**	Tech Lead	**9 (Medium)**
R-02	People & Skills	Skill gap inside the team across new tools	4	4	**16 (High)**	Tech Lead	**9 (Medium)**
R-03	People & Skills	Dependence on one or two members for reviews/knowledge	3	5	**15 (High)**	Tech Lead	**8 (Medium)**
R-04	Technical	Framework learning curve (Spring Boot 3.5 & JPA)	3	3	**9 (Medium)**	Tech Lead	**6 (Medium)**
R-05	Technical	SQLite limitations (decimals, single writer, dialect)	2	4	**8 (Medium)**	Tech Lead	**4 (Low)**
R-06	Technical	Money-integrity or parallel concurrency bugs	3	4	**12 (High)**	Tech Lead	**8 (Medium)**
R-07	Requirements	Ambiguous or changing project requirements	4	3	**12 (High)**	QA Lead	**6 (Medium)**
R-08	Quality & Testing	Flaky or fragile UI automation (Selenium 4)	4	3	**12 (High)**	QA Lead	**6 (Medium)**
R-09	Environment	Different developer laptops give different results	3	3	**9 (Medium)**	QA Lead	**4 (Low)**
R-10	Security	Spring Boot 3.5 no longer receives free security updates	3	2	**6 (Medium)**	Tech Lead	**4 (Low)**
R-11	Security	Security weaknesses in application endpoints/pages	3	4	**12 (High)**	Tech Lead	**6 (Medium)**
R-12	Security	Vulnerable or malicious third-party Maven dependencies	3	3	**9 (Medium)**	Tech Lead	**4 (Low)**
R-13	Security	Secrets, credentials, or SQLite DB committed to Git	2	4	**8 (Medium)**	Tech Lead	**3 (Low)**
R-14	Environment	Loss of local code work or test data	2	4	**8 (Medium)**	Whole Team	**3 (Low)**
R-15	Process	Git merge conflicts and a broken main branch	4	3	**12 (High)**	Tech Lead	**4 (Low)**
R-16	Quality & Testing	Low code maintainability or low test coverage	3	3	**9 (Medium)**	QA Lead	**6 (Medium)**

ID	Category	Risk Title	L	I	Inherent	Owner	Residual
R-17	People & Skills	Limited availability (university exams & other courses)	4	4	**16 (High)**	Whole Team	**9 (Medium)**
R-18	People & Skills	Unclear task ownership or fragmented communication	3	3	**9 (Medium)**	Whole Team	**4 (Low)**
R-19	Delivery	Live demo failure on final presentation day	3	5	**15 (High)**	Tech Lead	**6 (Medium)**
R-20	Delivery	Instructor expectations or named deliverables missed	3	4	**12 (High)**	QA Lead	**3 (Low)**
R-21	Schedule & Scope	Task effort estimation errors on complex features	4	3	**12 (High)**	Tech Lead	**6 (Medium)**
R-22	Environment	Tool access and free-plan limits (GitHub, Jira)	2	2	**4 (Low)**	Tech Lead	**2 (Low)**
R-23	External	Published Egyptian market figures change before demo	2	2	**4 (Low)**	QA Lead	**2 (Low)**
R-24	Quality & Testing	Code committed that the team cannot fully explain	3	4	**12 (High)**	Tech Lead	**6 (Medium)**

Part 7. Detailed Risk Cards (R-01 to R-24)

Comprehensive analysis for each of the 24 risks, specifying root cause, impact consequence, early-warning indicators, preventive mitigation controls, Plan B contingency, assigned owner, residual score, and self-contained domain scope.

R-01 — Scope too large for the available time

Category & Owner	Category: Schedule & Scope	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Monitored Weekly
Risk Description	The system encompasses multiple interconnected modules (account hierarchy, atomic transfers with dynamic fees, paginated statements, CSV export, admin dashboard, scheduled interest jobs, Spring Security, and audit logs) within an 8-week timeline. Attempting everything simultaneously risks leaving core features half-finished.
Root Cause & Consequence	• Cause: Ambitious functional scope combined with a five-member team balancing university coursework. • Consequence: Core 'Must-have' features missing or unstable during the final demo; rushed testing and lower code quality.
Inherent vs. Residual Score	**Inherent Risk: 4 × 4 = 16 (High) ---> Residual Risk (After Mitigation): 3 × 3 = 9 (Medium)**
Early-Warning Indicators	Fewer than 60% of planned Jira sprint items completed for two consecutive weeks, or any 'Must-have' feature still unbuilt at the start of Week 5.
Preventive Mitigation Actions	Apply strict MoSCoW prioritization (Must / Should / Could). Define the Minimum Viable Product (MVP) as all Must requirements. Build vertical end-to-end slices one feature at a time, check scope weekly against the 8-week schedule, and pre-approve the feature descoping order in Part 8.

Contingency Plan (Plan B)	Activate the Descoping Order (Part 8.2): cut Could items first (rate limiting), followed by lower-priority Should items (UI polish, CSV export, advanced admin filters), and record the decision in the weekly meeting log.
Affected System Scope	All Must-have Functional Modules & 8-Week Execution Schedule

R-02 — Skill gap inside the team

Category & Owner	Category: People & Skills	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Mitigation Active
Risk Description	Team members possess varying levels of prior experience, and several core technologies (Spring Boot 3.5, Spring Data JPA, Spring Security, Selenium 4 WebDriver, Postman automation) are new to multiple members.
Root Cause & Consequence	• Cause: Mixed technical background across the five-member team and multiple new tools introduced simultaneously. • Consequence: Slow implementation velocity, recurring bugs, and experienced members becoming review bottlenecks.
Inherent vs. Residual Score	**Inherent Risk: 4 × 4 = 16 (High) ---> Residual Risk (After Mitigation): 3 × 3 = 9 (Medium)**
Early-Warning Indicators	A Jira task remains blocked for more than 2 days; Pull Requests are repeatedly rejected for the same structural mistake; or a member cannot explain their own code.
Preventive Mitigation Actions	Provide a working reference vertical slice (deposit and withdrawal pattern) in the repository for members to follow. Break work into small 1–2 day tasks, pair-program on each member's first Pull Request, write explanatory code-review comments, and maintain shared setup/troubleshooting notes.
Contingency Plan (Plan B)	Reassign critical-path tasks (such as atomic transfers and security filter chains) to the Tech Lead or Core Logic Engineer, while keeping the original member as a co-reviewer in training and reducing their task complexity.
Affected System Scope	Team Task Assignment, Peer-Review Matrix & Codebase Onboarding

R-03 — Dependence on one or two members for reviews and critical knowledge

Category & Owner	Category: People & Skills	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Monitored Weekly
Risk Description	Architectural knowledge, security configuration, and Pull Request review responsibilities can concentrate around the Tech Lead or QA Lead. If a key member is unavailable due to illness or exams, progress can stall.
Root Cause & Consequence	• Cause: Uneven prior experience with Spring Boot architecture and security testing across the team. • Consequence: Pull Request merge queue stalls; critical path features are delayed; architectural context is bottlenecked.
Inherent vs. Residual Score	**Inherent Risk: 3 × 5 = 15 (High) ---> Residual Risk (After Mitigation): 2 × 4 = 8 (Medium)**
Early-Warning Indicators	Pull Requests waiting more than 48 hours for review, or a key lead unavailable for more than 3 consecutive days.
Preventive Mitigation Actions	Document all architectural rules, database configurations, and error codes clearly in the repository README and self-contained specifications. Assign a named Secondary Reviewer for every module in the Peer-Review Matrix so at least two members understand each subsystem.

Contingency Plan (Plan B)	Delegate non-security Pull Request approvals to the designated Secondary Reviewer; temporarily freeze new feature additions and focus the team on unit/integration testing until the key member returns.
Affected System Scope	Layered Architecture, Git Pull Request Workflow & Peer-Review Matrix

R-04 — Framework learning curve (Spring Boot and JPA)

Category & Owner	Category: Technical	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Spring Boot annotations, declarative @Transactional rollback rules, Spring Security filter chains, and JPA entity lifecycle mappings behave in ways that can be non-obvious to developers new to the ecosystem.
Root Cause & Consequence	• Cause: Heavy reliance on declarative framework annotations and convention-over-configuration patterns. • Consequence: Hours wasted debugging startup or mapping exceptions instead of implementing banking business rules.
Inherent vs. Residual Score	**Inherent Risk: 3 × 3 = 9 (Medium) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	More than one team member stuck on Spring configuration, bean wiring, or Hibernate mapping errors within the same week.
Preventive Mitigation Actions	Deliver a pre-configured, working Spring Boot 3.5.x project skeleton with SQLite, Security, and a sample vertical slice already wired. Require members to share the first 40 lines of any stack trace in the team chat immediately if blocked for over 2 hours, and follow established repository patterns.
Contingency Plan (Plan B)	The Tech Lead resolves the framework configuration issue directly via pair debugging and documents the fix in the README troubleshooting guide.
Affected System Scope	Spring Boot 3.5.x Skeleton, JPA Entity Mappings & README Setup Guide

R-05 — SQLite limitations (no exact decimals, single writer, dialect support)

Category & Owner	Category: Technical	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Verified by Week-1 Architecture Spike
Risk Description	SQLite stores native decimal types as floating-point numbers (REAL), permits only one concurrent database writer at a time, and relies on the Hibernate community dialect.
Root Cause & Consequence	• Cause: Architectural choice of embedded single-file SQLite for zero-install portability across student laptops. • Consequence: Floating-point rounding errors in EGP balances, 'SQLITE_BUSY / database is locked' exceptions under parallel requests, or ORM dialect quirks.
Inherent vs. Residual Score	**Inherent Risk: 2 × 4 = 8 (Medium) ---> Residual Risk (After Mitigation): 1 × 4 = 4 (Low)**
Early-Warning Indicators	Any 'database is locked' exception during testing, any 1-piaster rounding discrepancy in balance calculations, or a failing concurrency test.
Preventive Mitigation Actions	Store all monetary amounts as integer whole piasters (BIGINT, where 1 EGP = 100 piasters) mapped to Java BigDecimal (scale 2, HALF_UP) via a JPA AttributeConverter, and store rates as integer basis points. Configure SQLite with Write-Ahead Logging (PRAGMA journal_mode=WAL), PRAGMA busy_timeout=5000, BEGIN IMMEDIATE transactions, and a single-connection HikariCP pool (maximum-pool-size=1). Verify via an early Week 1 spike and automated parallel withdrawal test (TC-C-01).

Contingency Plan (Plan B)	If SQLite exhibits unresolvable locking or dialect defects before Week 4, switch the Spring Data JPA datasource configuration to embedded H2 (in PostgreSQL/MySQL compatibility mode) or MySQL without altering domain logic.
Affected System Scope	Monetary Correctness (BIGINT Piasters), Atomic Transactions & SQLite WAL Concurrency

R-06 — Money-integrity or concurrency bugs

Category & Owner	Category: Technical	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Multi-step operations such as inter-customer transfers, dynamic fee deductions, overdraft checks, and daily/monthly velocity limits can create or destroy money, bypass limits, or leave partial updates if not strictly atomic.
Root Cause & Consequence	• Cause: Complex financial domain rules combined with potential simultaneous requests on the same account. • Consequence: Corrupted account balances, mismatch between account balance and ledger sum, or failed acceptance tests.
Inherent vs. Residual Score	**Inherent Risk: 3 × 4 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 4 = 8 (Medium)**
Early-Warning Indicators	Account balance disagrees with the sum of ledger rows in any test; intermittent transfer test failures; or business calculation logic found inside a web controller.
Preventive Mitigation Actions	Execute every money operation inside a single @Transactional boundary where any unchecked domain exception rolls back all changes. Keep all calculation rules inside the Domain and Service layers using BigDecimal (HALF_UP). Enforce append-only ledger rows (TRANSFER_OUT, TRANSFER_IN, and FEE sharing one reference ID). Require JUnit 5 unit tests and multi-threaded concurrency tests (TC-C-01) for every financial rule, reviewed by the Tech Lead and QA Lead.
Contingency Plan (Plan B)	Halt merging of new financial features immediately; write a failing reproduction test, fix the atomicity or rounding defect, and re-run the entire JUnit and Postman regression suite.
Affected System Scope	Core Transaction Engine (Deposit, Withdraw, Transfer, Fee Triplet & Velocity Limits)

R-07 — Ambiguous or changing requirements

Category & Owner	Category: Requirements	Risk Owner: QA Lead (Somia Rashad)	Status: Mitigated — Standardized in Requirement Gathering Specification
Risk Description	Business rules or edge cases (such as exact error codes when multiple rules fail or external stakeholder expectations) may be interpreted differently by developers and testers.
Root Cause & Consequence	• Cause: Requirements authored prior to code implementation; potential new requests during instructor milestone reviews. • Consequence: Code rework, automated tests failing against developer assumptions, and late-stage team disagreements.
Inherent vs. Residual Score	**Inherent Risk: 4 × 3 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	A developer and QA tester disagree on the expected error code or status of an operation, or the instructor requests a scope adjustment.

Preventive Mitigation Actions	Perform static testing of all requirements in Weeks 1–2 and consolidate all User Stories, Functional/Non-Functional Requirements, all 29 Business Rules, and the standardized Error Code Catalogue into a single self-contained specification (e.g., explicitly standardizing MIN_BALANCE_VIOLATION for Savings minimum balance breaches). Log any requirement change in Jira with version tracking.
Contingency Plan (Plan B)	Freeze the agreed interpretation in the Requirement Gathering specification, update affected test cases and domain code within 24 hours, and notify the entire team.
Affected System Scope	Self-Contained Requirement Gathering Document, Business Rules & Error Code Catalogue

R-08 — Flaky or fragile UI automation (Selenium)

Category & Owner	Category: Quality & Testing	Risk Owner: QA Lead (Somia Rashad)	Status: Open
Risk Description	Automated Selenium WebDriver tests can fail intermittently if HTML element IDs change during UI styling, if scripts run before DOM rendering finishes, or if tests share mutated database state.
Root Cause & Consequence	• Cause: Ongoing frontend template edits, asynchronous page rendering, and shared seed data across test runs. • Consequence: False-positive test failures, wasted hours debugging test scripts in Week 6, and unreliable test evidence.
Inherent vs. Residual Score	**Inherent Risk: 4 × 3 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	A Selenium test passes and fails on consecutive runs without code changes, or UI tests break after a CSS/Thymeleaf update.
Preventive Mitigation Actions	Mandate permanent, unique HTML element IDs across all Thymeleaf forms and banners, frozen at the end of Week 4. Structure Selenium tests using the Page Object Model (POM) and explicit WebDriverWait conditions (banning Thread.sleep). Reset the database to deterministic seed data before every automated test suite run.
Contingency Plan (Plan B)	Quarantine any persistently flaky UI test, log a Jira defect, and verify the underlying feature via MockMvc/Postman API tests and structured manual testing until the UI locator is fixed.
Affected System Scope	Thymeleaf Stable DOM Element IDs, UI Freeze (Week 4) & Selenium 4 Page Object Suite

R-09 — Different laptops give different results

Category & Owner	Category: Environment & Tooling	Risk Owner: QA Lead (Somia Rashad)	Status: Open
Risk Description	Team members work across different Windows/Linux environments, terminal shells (PowerShell vs. Bash), and local JDK installations, which can cause build or path discrepancies.
Root Cause & Consequence	• Cause: Distributed development across five personal workstations without a centralized cloud test server. • Consequence: 'Works on my machine' failures and delays during peer reviews or test execution.
Inherent vs. Residual Score	**Inherent Risk: 3 × 3 = 9 (Medium) ---> Residual Risk (After Mitigation): 2 × 2 = 4 (Low)**
Early-Warning Indicators	A team member cannot build or start the application cleanly after cloning the repository.
Preventive Mitigation Actions	Standardize on JDK 17+, commit the Maven Wrapper (./mvnw and mvnw.cmd), use the embedded single-file SQLite database requiring zero external server installation, and document exact PowerShell and Bash startup/test commands in the repository README.

Contingency Plan (Plan B)	Conduct a remote pair-debugging session with the Tech Lead to align the member's local environment variables and document the fix in the README.
Affected System Scope	Portability, Maven Wrapper, Embedded SQLite & Cross-Platform README Instructions

R-10 — Spring Boot 3.5 no longer receives free security updates

Category & Owner	Category: Security	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Accepted for Local Demonstration
Risk Description	The Spring Boot 3.x branch reached the end of free commercial security support in June 2026, meaning newly discovered framework CVEs will not receive free upstream patches.
Root Cause & Consequence	• Cause: Selection of Spring Boot 3.5.x for ecosystem stability, Hibernate SQLite dialect compatibility, and extensive documentation. • Consequence: Potential vulnerability scanner warnings in third-party transitive dependencies; a point requiring technical defense during the final presentation.
Inherent vs. Residual Score	**Inherent Risk: 3 × 2 = 6 (Medium) ---> Residual Risk (After Mitigation): 2 × 2 = 4 (Low)**
Early-Warning Indicators	OWASP Dependency-Check or GitHub Dependabot reports a High or Critical CVE in a transitive Spring library.
Preventive Mitigation Actions	Run the application strictly as a local offline demonstration (never exposed to the public internet). Execute an OWASP Dependency-Check scan prior to final delivery, upgrade any patchable transitive library, and explicitly document the known lifecycle limitation and the production upgrade path to Spring Boot 4.1 in the security documentation.
Contingency Plan (Plan B)	If a critical exploitable dependency flaw is flagged and time permits, upgrade the parent POM version; otherwise, document the risk acceptance and local-only mitigation for the academic defense.
Affected System Scope	Dependency Vulnerability Scanning (OWASP) & Security Documentation

R-11 — Security weaknesses in the application

Category & Owner	Category: Security	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open — Automated Security Tests Assigned
Risk Description	Web banking endpoints and forms could suffer from Broken Access Control, Insecure Direct Object Reference (IDOR), brute-force login attacks, SQL Injection, XSS, or CSRF vulnerabilities.
Root Cause & Consequence	• Cause: Multiple user roles (CUSTOMER, ADMIN), numerous financial endpoints, and developers writing new controller/service code. • Consequence: Unauthorized access to another customer's account or statement, privilege escalation, or failed security test cases.
Inherent vs. Residual Score	**Inherent Risk: 3 × 4 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	Any failing security test case (IDOR, role check, CSRF, or lockout), or a Pull Request review uncovering a missing ownership check.
Preventive Mitigation Actions	Enforce server-side role and ownership verification on every request via centralized service checks (returning HTTP 404 ACCOUNT_NOT_FOUND on cross-customer access to prevent IDOR enumeration). Enforce BCrypt hashing, 5-attempt / 15-minute lockout, 15-minute session idle timeout, CSRF tokens on all web forms, parameterized JPA queries, and Thymeleaf output escaping (th:text). Verify with dedicated automated security tests.

Contingency Plan (Plan B)	Treat any security flaw as a Critical defect: block feature merges, implement the server-side fix immediately, add an automated regression test, and re-run the full security test suite.
Affected System Scope	Spring Security Architecture, RBAC, IDOR 404 Protection & OWASP Controls

R-12 — Vulnerable or malicious dependencies

Category & Owner	Category: Security	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Third-party open-source libraries pulled via Maven could contain known security vulnerabilities or supply-chain flaws.
Root Cause & Consequence	• Cause: Transitive Maven dependencies required for web, persistence, and testing frameworks. • Consequence: Security scanner alerts or unstable runtime behavior.
Inherent vs. Residual Score	**Inherent Risk: 3 × 3 = 9 (Medium) ---> Residual Risk (After Mitigation): 2 × 2 = 4 (Low)**
Early-Warning Indicators	GitHub Dependabot alert or OWASP Dependency-Check report flagging a vulnerable JAR.
Preventive Mitigation Actions	Use only official Maven Central releases with pinned versions in pom.xml, enable GitHub Dependabot alerts on the repository, require Tech Lead review for any new dependency addition in Pull Requests, and run OWASP Dependency-Check before Milestone 4.
Contingency Plan (Plan B)	Upgrade or replace the vulnerable dependency version in pom.xml; if no patch exists, document the mitigation in the final security report.
Affected System Scope	Maven POM Dependency Management & OWASP Dependency-Check

R-13 — Secrets or databases committed to Git

Category & Owner	Category: Security	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Developer credentials, environment files (.env), or local SQLite database files (.db / .sqlite) could be accidentally staged and pushed to the shared GitHub repository.
Root Cause & Consequence	• Cause: Fast local commits or unconfigured .gitignore rules. • Consequence: Exposed bootstrap credentials, binary merge conflicts on database files, and polluted Git history.
Inherent vs. Residual Score	**Inherent Risk: 2 × 4 = 8 (Medium) ---> Residual Risk (After Mitigation): 1 × 3 = 3 (Low)**
Early-Warning Indicators	A Pull Request file diff showing a .db, .sqlite, or .env file, or a secret-scanner warning.
Preventive Mitigation Actions	Configure a strict .gitignore file from Day 1 excluding *.db, *.sqlite, *.sqlite-wal, *.sqlite-shm, and .env files. Provision the initial admin account from environment variables (with fictional demo credentials restricted to the dev/test profiles). Inspect changed files in every Pull Request review.
Contingency Plan (Plan B)	Immediately rotate any exposed credential, purge the file from Git history using git filter-repo / reset, and enforce branch push checks.
Affected System Scope	Repository .gitignore Standards, Environment Variables & Git Hygiene

R-14 — Loss of work or data

Category & Owner	Category: Environment & Tooling	Risk Owner: Whole Team	Status: Open
Risk Description	A workstation hardware failure, accidental directory deletion, or corrupted local SQLite database could erase days of unpushed code or documentation.

Root Cause & Consequence	• Cause: Keeping code or documentation changes exclusively on a single local laptop for multiple days. • Consequence: Significant rework and missed weekly milestone targets.
Inherent vs. Residual Score	**Inherent Risk: 2 × 4 = 8 (Medium) ---> Residual Risk (After Mitigation): 1 × 3 = 3 (Low)**
Early-Warning Indicators	A team member has active work that has not been pushed to a remote GitHub branch for more than 48 hours.
Preventive Mitigation Actions	Mandate daily git push of work-in-progress feature branches to GitHub. Keep all official project documentation versioned inside the repository milestone folders, and make the SQLite database 100% reproducible on startup via deterministic dev seed data.
Contingency Plan (Plan B)	Restore the latest branch state from GitHub and regenerate the local SQLite database automatically using the dev seed profile.
Affected System Scope	Git Backup Discipline, Milestone Folders & Reproducible Seed Data

R-15 — Merge conflicts and a broken main branch

Category & Owner	Category: Process	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Multiple developers editing shared files (such as application.yml, layout templates, or common DTOs) or pushing untested code directly to main can break the project build.
Root Cause & Consequence	• Cause: Parallel development across five team members on a shared full-stack repository. • Consequence: Broken builds blocking other team members, lost code during conflict resolution, and wasted time.
Inherent vs. Residual Score	**Inherent Risk: 4 × 3 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 2 = 4 (Low)**
Early-Warning Indicators	Compilation or test failures on the main branch, or recurring Git merge conflicts in shared files.
Preventive Mitigation Actions	Enforce branch protection on main: prohibit direct pushes from team members, work in short-lived feature/chore branches, pull from main before starting work and before opening a Pull Request, run './mvnw test' locally before pushing, and require at least one peer review approval before merging.
Contingency Plan (Plan B)	Revert the breaking merge commit immediately on main, resolve conflicts cleanly on the feature branch with Tech Lead assistance, re-verify all tests, and merge via a clean Pull Request.
Affected System Scope	Git Branching Strategy, Pull Request Protocol & Main Branch Protection

R-16 — Low code quality or low test coverage

Category & Owner	Category: Quality & Testing	Risk Owner: QA Lead (Somia Rashad)	Status: Open
Risk Description	Under schedule pressure, code may be written that passes basic happy-path clicks during a demo but lacks unit tests, violates layered architecture, or contains hidden edge-case bugs.
Root Cause & Consequence	• Cause: Rushing feature completion in Weeks 3–4 without writing simultaneous unit and boundary tests. • Consequence: Missed KPI targets (KPI-02 100% requirement coverage), weak testing evidence for the instructor, and regressions in Week 6.
Inherent vs. Residual Score	**Inherent Risk: 3 × 3 = 9 (Medium) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	JaCoCo code coverage drops below target, or a 'Must-have' requirement lacks a linked test case in the RTM.

Preventive Mitigation Actions	Enforce a strict Definition of Done for every Pull Request: layered architecture respected (zero business logic in controllers/templates), JUnit 5 unit tests included, SonarLint checks clean, and every requirement mapped in the Requirement Traceability Matrix (RTM).
Contingency Plan (Plan B)	Dedicate Week 6 exclusively to static analysis, gap testing, and defect resolution, prioritizing test cases for high-risk financial and security rules first.
Affected System Scope	KPI-02 (100% Coverage), RTM, JUnit 5 Suite & SonarLint Static Analysis

R-17 — Limited availability (exams, other courses and training)

Category & Owner	Category: People & Skills | Risk Owner: Whole Team | Status: Open
Risk Description	All team members are university students balancing academic exams, faculty lectures, and parallel obligations, causing weekly available hours to fluctuate.
Root Cause & Consequence	• Cause: Competing university exam schedules and academic deadlines during the 8-week project window. • Consequence: Uneven weekly sprint velocity and potential delays in assigned module deliverables.
Inherent vs. Residual Score	**Inherent Risk: 4 × 4 = 16 (High) ---> Residual Risk (After Mitigation): 3 × 3 = 9 (Medium)**
Early-Warning Indicators	A member reports less than 50% of their planned weekly capacity, or misses two consecutive weekly sync meetings.
Preventive Mitigation Actions	Share university exam and unavailability dates upfront during sprint planning. Deconstruct work into small, independent 1–2 day tasks, cross-train secondary reviewers as backups, and preserve Week 8 as a dedicated schedule buffer.
Contingency Plan (Plan B)	Reallocate urgent tasks across available team members for that week and apply the agreed feature Descoping Order (Part 8.2) if total team capacity drops.
Affected System Scope	Weekly Sprint Capacity Planning, Cross-Training & Week 8 Buffer

R-18 — Unclear ownership or poor communication

Category & Owner	Category: People & Skills | Risk Owner: Whole Team | Status: Open
Risk Description	Two members might accidentally implement the same component, a critical task might be overlooked assuming someone else owns it, or key technical decisions might get buried in informal chat messages.
Root Cause & Consequence	• Cause: Informal communication across five remote team members without centralized tracking. • Consequence: Duplicated effort, missing integration pieces, or mismatched API/UI contracts.
Inherent vs. Residual Score	**Inherent Risk: 3 × 3 = 9 (Medium) ---> Residual Risk (After Mitigation): 2 × 2 = 4 (Low)**
Early-Warning Indicators	A scheduled task has no assigned owner in Jira for more than 48 hours, or the same requirement question is debated repeatedly.
Preventive Mitigation Actions	Enforce the Task Assignment & Peer-Review Matrix (one Primary Owner and one Secondary Reviewer per deliverable). Use the Jira board as the single source of truth for task status, hold structured weekly meetings with written notes, and record all rules in the self-contained specification documents.
Contingency Plan (Plan B)	The Tech Lead and QA Lead assign explicit task ownership on the spot during the sync check and log it immediately in Jira.
Affected System Scope	Task Assignment & Peer-Review Matrix, Jira Board & Weekly Sync Governance

R-19 — Demo failure on presentation day

Category & Owner	Category: Delivery	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	During the final live defense, the demonstration could fail due to corrupted database state, a port conflict, a laptop hardware/display glitch, or an unscripted input triggering an unhandled bug.
Root Cause & Consequence	• Cause: Inherent fragility of live software demonstrations under presentation time pressure. • Consequence: A well-engineered and thoroughly tested project appears broken in front of the evaluator.
Inherent vs. Residual Score	**Inherent Risk: 3 × 5 = 15 (High) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	Any application crash, seed-data mismatch, or script hiccup during the Week 7 dry-run rehearsals.
Preventive Mitigation Actions	Run the demonstration 100% locally offline (zero dependence on venue Wi-Fi). Conduct two full dress rehearsals in Week 7 following a written step-by-step demo script. Include a one-command database reset script that loads clean, deterministic seed data before the demo. Have the complete project installed and verified on a second backup laptop, plus a pre-recorded high-resolution backup video of the full demo flow.
Contingency Plan (Plan B)	Switch immediately to the backup laptop or play the recorded demonstration video while presenting the live test execution reports (JUnit, Postman, Selenium, and Jira RTM) as proof of quality.
Affected System Scope	Week 7–8 Demo Rehearsals, Deterministic Seed Reset, Backup Laptop & Video

R-20 — Instructor expectations not covered (named deliverables)

Category & Owner	Category: Delivery	Risk Owner: QA Lead (Somia Rashad)	Status: Mitigated — All Milestone Deliverables Standardized
Risk Description	The track instructor evaluates specific mandatory documentation and testing artifacts (Project Proposal, Project Plan, Task Assignment & Roles, Risk Assessment & Mitigation Plan, KPIs, Requirement Gathering, Master Test Plan & RTM, and Automated Test Suites).
Root Cause & Consequence	• Cause: Misalignment between internal team artifacts and the formal grading rubric deliverables. • Consequence: Loss of evaluation marks despite having a working software application.
Inherent vs. Residual Score	**Inherent Risk: 3 × 4 = 12 (High) ---> Residual Risk (After Mitigation): 1 × 3 = 3 (Low)**
Early-Warning Indicators	A milestone review reveals that a required document section or testing artifact is missing or incomplete.
Preventive Mitigation Actions	Maintain a dedicated folder structure per milestone (milestone-1/, milestone-2/, etc.) containing every named deliverable in a self-contained, standardized format. Verify deliverable completeness against the track rubric before each milestone submission.
Contingency Plan (Plan B)	Author and submit any requested supplementary artifact or revision within 48 hours of instructor feedback.
Affected System Scope	Milestone Repository Structure & Deliverable Quality Sign-Off

R-21 — Estimation errors

Category & Owner	Category: Schedule & Scope	Risk Owner: Tech Lead (Muhammad Sayyid)	Status: Open
Risk Description	Complex technical tasks—such as atomic inter-customer transfers with fee triplets, scheduled interest jobs, or Selenium Page Object suites—may take longer than initially estimated.

Root Cause & Consequence	• Cause: Lack of historical velocity data for this specific five-member team and tech stack. • Consequence: Spillover of development tasks into Weeks 5–6, compressing the time available for regression testing.
Inherent vs. Residual Score	**Inherent Risk: 4 × 3 = 12 (High) ---> Residual Risk (After Mitigation): 3 × 2 = 6 (Medium)**
Early-Warning Indicators	An active development task exceeds twice its original day estimate in Jira.
Preventive Mitigation Actions	Estimate tasks in days rather than hours and include a 30% buffer for unfamiliar tools. Execute the highest-risk technical spikes early (Week 1 SQLite WAL/piaster spike). Compare planned vs. actual completion weekly in Jira (KPI-01) and rebalance the upcoming sprint.
Contingency Plan (Plan B)	Defer the lowest-priority 'Could' or 'Should' item out of the current week using the Descoping Order (Part 8.2) to protect core 'Must' features and testing time.
Affected System Scope	KPI-01 (Schedule Adherence), Sprint Estimation & Descoping Order

R-22 — Tool access and plan limits (GitHub organisation, Jira)

Category & Owner	Category: Environment & Tooling | Risk Owner: Tech Lead (Muhammad Sayyid) | Status: Open
Risk Description	Certain repository governance features (such as advanced branch protection rules on private free-tier GitHub organizations or Jira user/automation limits) may be restricted on free academic/community plans.
Root Cause & Consequence	• Cause: Zero-budget academic constraint requiring 100% free-tier development and management tools. • Consequence: Automated tool enforcement of certain workflow rules may be unavailable.
Inherent vs. Residual Score	**Inherent Risk: 2 × 2 = 4 (Low) ---> Residual Risk (After Mitigation): 1 × 2 = 2 (Low)**
Early-Warning Indicators	A required GitHub branch setting or Jira board permission is greyed out or restricted by plan limits.
Preventive Mitigation Actions	Configure the GitHub organization, repository, and Jira board in Week 1. Grant administrative access to both the Tech Lead and QA Lead. Where a free plan restricts automated enforcement, use a public repository or enforce the rule via mandatory team Pull Request checklists.
Contingency Plan (Plan B)	Switch repository visibility or apply a manual PR review sign-off checklist enforced by the Tech Lead.
Affected System Scope	GitHub Repository Governance & Jira Free-Tier Configuration

R-23 — Market figures change (limits, fees, interest rates)

Category & Owner	Category: External | Risk Owner: QA Lead (Somia Rashad) | Status: Open
Risk Description	Real-world Egyptian banking benchmarks (Central Bank policy rates, InstaPay transfer fee tiers, or bank savings rates) referenced in the project may change before the final presentation.
Root Cause & Consequence	• Cause: External macroeconomic or regulatory updates by the Central Bank of Egypt or local payment networks. • Consequence: Hardcoded financial rates or limits in code and test cases could appear outdated or inconsistent.
Inherent vs. Residual Score	**Inherent Risk: 2 × 2 = 4 (Low) ---> Residual Risk (After Mitigation): 1 × 2 = 2 (Low)**
Early-Warning Indicators	Public announcements of updated transfer fee caps or savings interest rates prior to Week 8.

Preventive Mitigation Actions	Externalize all numerical business parameters (Savings 17% rate, FD 17.00%/17.75% rates, 0.1% transfer fee, 0.50/20.00 EGP fee bounds, and 70k/120k/400k EGP withdrawal limits) into application.yml rather than hardcoding magic numbers in Java classes.
Contingency Plan (Plan B)	Update application.yml and the corresponding seed/test constants in a single configuration commit if required before the final defense.
Affected System Scope	Externalized Business Rule Configuration (application.yml)

R-24 — Code that the team does not fully understand

Category & Owner	Category: Quality & Testing | Risk Owner: Tech Lead (Muhammad Sayyid) | Status: Open
Risk Description	Code snippets adapted from framework documentation, boilerplate generators, or written by another teammate may not be fully understood by the member presenting that module during the final oral defense.
Root Cause & Consequence	• Cause: Fast development pace, use of reference examples, or insufficient walkthroughs during peer reviews. • Consequence: Inability to answer instructor questions during the final defense, hidden logic bugs, and academic integrity concerns.
Inherent vs. Residual Score	**Inherent Risk: 3 × 4 = 12 (High) ---> Residual Risk (After Mitigation): 2 × 3 = 6 (Medium)**
Early-Warning Indicators	A team member cannot explain line-by-line how a class, annotation, or test in their assigned module works during a weekly sync or PR review.
Preventive Mitigation Actions	Enforce the Peer-Review Matrix: every Pull Request must be reviewed by a teammate who reads and runs the code. Each member writes the JUnit/automated tests for their own module and conducts a brief code walkthrough for the team before Week 7 rehearsals.
Contingency Plan (Plan B)	Hold dedicated team code-walkthrough sessions in Week 7 and refactor or simplify any overly complex code construct that a member cannot confidently explain.
Affected System Scope	Peer-Review Matrix, Code Walkthroughs & Final Oral Defense Readiness

Part 8. Contingency Plans & Feature Descoping Order

Pre-approved Plan B operational scenarios and the strict priority order for cutting non-essential scope if schedule pressure arises.

8.1 Plan B Contingency Scenarios

Trigger Scenario	Early-Warning Signal	Actionable Fallback (Plan B)	Decision Authority
SQLite proves unreliable under concurrency	Recurring 'database is locked' errors, piaster rounding mismatches, or failing concurrency test TC-C-01	Switch Spring Data JPA configuration to embedded H2 or MySQL before Week 4 and re-run the full JUnit test suite.	Whole Team (Led by Tech Lead)
A key lead is unavailable for > 3 days	Pull Requests waiting more than 48 hours for review	Designated Secondary Reviewer approves non-security PRs; freeze new feature additions and focus on testing existing slices.	Whole Team

Trigger Scenario	Early-Warning Signal	Actionable Fallback (Plan B)	Decision Authority
A 'Must-have' feature risks missing the deadline	Any Must requirement still unbuilt at the start of Week 5	Immediately apply the Descoping Order (Section 8.2) to cut Could and Should items and reallocate developer hours to the Must feature.	Whole Team (Instructor Notified)
Selenium UI automation suite is unstable/flaky	Same WebDriver test passes and fails intermittently on re-runs	Quarantine flaky UI scripts; keep stable core UI flows running and rely on MockMvc, Postman API tests, and documented manual tests for the affected screen.	QA Lead & Tech Lead
A team member is absent for an extended period	Missed weekly meetings and zero commits for over 5 days	Redistribute the member's Must-have tasks by priority across the remaining team and cut lower-priority admin/reporting Should items first.	Whole Team
Primary demo laptop or environment fails on defense day	Any hardware, port, or database glitch during Week 7 rehearsals	Switch immediately to the pre-configured secondary backup laptop or play the recorded full-demo backup video alongside live test reports.	Tech Lead & QA Lead
High/Critical vulnerability found in a Maven dependency	OWASP Dependency-Check or GitHub Dependabot alert	Upgrade or replace the affected library in pom.xml; if no upstream fix exists (e.g., Spring Boot 3.5 EOL), document local-only containment and the Spring Boot 4.1 upgrade path.	Tech Lead
Late requirement clarification or change from instructor	Instructor feedback during milestone review	Perform a 24-hour impact analysis, update the Requirement Gathering document and RTM, and trade off an equivalent Should item if needed.	Whole Team

8.2 Feature Descoping Order (What to Cut First if Time Runs Out)

Core 'Must-have' requirements are strictly protected and are never cut without a formal team decision and instructor notification. If schedule pressure requires reducing scope, the team cuts features strictly in the order below (starting from Cut #1 at the top):

Cut Order	Feature / Capability to Descope	Priority & Affected Module Scope
Cut #1 (First)	In-memory rate limiting on money endpoints and optional JWT token mode	Could Priority — Security Abuse Prevention Enhancement
Cut #2	Extra UI visual polish and secondary wording refinements on the transfer confirmation screen	Should Priority — Frontend Transfer Review Polish
Cut #3	Exporting filtered customer account statements as a downloadable CSV file	Should Priority — Statement CSV Export Pipeline

Cut Order	Feature / Capability to Descope	Priority & Affected Module Scope
Cut #4	Advanced admin tools: audit-log web viewer, manual interest trigger button, balance-range account search, and richest customer report	Should Priority — Secondary Admin Reporting & Manual Job Trigger Views
Cut #5	Filtering customer account statements by start/end date range and transaction type	Should Priority — Statement Date & Type Filtering (keep paginated statement list)
Cut #6	HTTP Idempotency-Key header caching for duplicate POST retry protection	Should Priority — Transaction Retry Idempotency Cache
Cut #7	Automated background cron schedulers for monthly savings interest and fixed deposit maturity	Should Priority — Background Cron Scheduling (keep interest calculation methods tested and invoked via service/test calls)
Cut #8 (Last)	Automated 5-attempt login lockout timer and 15-minute idle session auto-expiry	Should Priority — Advanced Session/Lockout Timers (only cut after BCrypt, RBAC, CSRF, and IDOR 404 checks are 100% verified)

8.3 Protected Core MVP Scope (Never Cut)

Protected Domain Area	Mandatory Protected Capabilities (100% Protected from Descoping)
Authentication & Access Control	Visitor registration, Customer/Admin login & logout, BCrypt password policy, server-side RBAC, environment-variable admin bootstrap, and HTTP 404 IDOR ownership protection.
Customer Profile & Dashboard	Viewing and editing customer full name and phone number, input validation rules, and the customer account overview dashboard with aggregate non-closed balance.
Core Account Hierarchy	Opening Savings (1,000 EGP min), Checking (-1,000 EGP overdraft), and Fixed Deposit (5,000 EGP min with term) accounts; 12-digit Luhn account numbers; closing zero-balance active accounts; and Admin account freeze/unfreeze with audit reason.
Atomic Transactions & Ledger	Deposits, withdrawals, own-account transfers (0 EGP fee), and inter-customer transfers (0.1% fee, min 0.50, max 20.00 EGP); single/daily/monthly withdrawal limits; append-only ledger rows; atomic rollback; piaster BIGINT storage; and deterministic error codes.
Statements & Core Admin	Paginated customer account statements (newest first with running balance), Admin bank-reserves dashboard, Admin customer partial-name search, and strict Admin least-privilege enforcement (admins cannot move money or read passwords).
Quality & Testing Evidence	Self-contained Requirement Gathering specification, Master Test Plan & RTM, JUnit 5 unit tests, Postman API test collection, Selenium 4 core flow tests, and Jira defect tracking reports.

Part 9. Risk Monitoring, Control & Escalation

Operational cadence for reviewing risks, escalating critical issues, and measuring risk management effectiveness.

9.1 Risk Review Cadence

Governance Activity	Frequency	Execution Details & Outputs
Weekly Team Risk Review	Weekly (15 minutes during team sync)	Review the Top 10 risks, check early-warning indicators, update Likelihood/Impact scores, and verify mitigation progress. Chaired by the Risk Coordinator (QA Lead).
Continuous Warning-Sign Watch	Daily / Continuous	Any team member who observes an early-warning indicator (e.g., a blocked task, a failing concurrency test, or a merge conflict) posts it immediately in the team chat and alerts the Risk Owner.
Milestone Risk Re-Scoring	End of Weeks 1, 4, 6, and 8	Full review of all 24 risks in the register; close risks that can no longer occur; confirm or update scope descoping decisions.
Jira Mitigation Tracking	Weekly	Every active mitigation action for a High risk is tracked as a Jira task with an assigned owner, due date, and the 'risk' label.

9.2 Escalation Rules

Trigger Situation	Mandatory Escalation Action & Timeline
Any risk score reaches Critical (17–25)	Escalate the same day to the entire team; agree on a remediation plan within 24 hours; notify the Track Instructor if any deliverable or milestone date is threatened.
A High risk lacks a named owner or mitigation date	The Risk Coordinator (Somia Rashad) and Tech Lead (Muhammad Sayyid) assign an owner and target date immediately during the weekly sync.
A planned mitigation action is overdue by > 1 week	Escalate in the weekly team meeting; either reallocate engineering support to complete the mitigation or activate the corresponding Plan B contingency.
A Plan B contingency scenario is activated	Record the activation in the weekly meeting log, inform all five team members, and update affected project documentation.

9.3 Risk Management KPIs

Risk Indicator	Target Benchmark	Measurement Method
Open Critical Risks	**0 at all times**	Direct count of Critical (17–25) risks in the Risk Register.
Open High Residual Risks	**0 by the end of Week 4**	Count of risks whose Residual Score remains in the High band (10–16).
On-Time Mitigation Completion	**85% or higher**	(Mitigation tasks completed on time / Total mitigation tasks due) × 100 in Jira.
Unassigned Risks	**0 (100% assigned)**	Verification that all 24 risks have an assigned owner role.

Risk Indicator	Target Benchmark	Measurement Method
Weekly Risk Reviews Held	**100% of project weeks**	Weekly team meeting records across the 8-week schedule.
Average Residual Risk Score Trend	**Decreasing or stable at each milestone**	Average of the 24 residual risk scores calculated at Weeks 1, 4, 6, and 8.

Part 10. Assumptions, Dependencies, Constraints & Glossary

Foundational project boundaries and definitions of risk and banking terminology used throughout this plan.

10.1 Project Assumptions, Dependencies & Constraints

Category	Defined Project Parameters
Assumptions	• The project is executed by the five-member team (Somia Rashad, Muhammad Sayyid, Aladdin Ahmed, Asmaa Mohamed, and Aghapy Antonious) over an 8-week schedule. • All development, testing, and management tools used are free/open-source, and the final demonstration runs locally using fictional seed data. • The Track Instructor reviews milestone deliverables and provides timely feedback.
Dependencies	• Continuous availability of GitHub, Jira, Maven Central, and a working JDK 17+ workstation for each team member. • Completion of the Week 1 SQLite WAL and piaster storage spike prior to building the transaction engine. • Freeze of Thymeleaf HTML element IDs at the end of Week 4 prior to full Selenium script execution in Week 5.
Constraints	• Fixed 8-week academic timeline with zero financial budget. • Concurrent university coursework and exam commitments across team members. • Spring Boot 3.5.x reached the end of free commercial security updates in June 2026 (mitigated via local-only execution and OWASP dependency scanning under Risk R-10).

10.2 Risk & Technical Glossary

Term	Definition in the Context of This Project
Inherent Risk	The initial severity score (Likelihood × Impact) of a risk before any preventive mitigation actions are applied.
Residual Risk	The remaining severity score (Likelihood × Impact) of a risk after preventive mitigation controls are implemented.
Mitigation	Proactive technical or process actions taken in advance to reduce the probability or impact of a risk.
Contingency Plan (Plan B)	The pre-agreed fallback action the team executes if a risk materializes despite mitigation efforts.
Early-Warning Indicator	An observable metric or signal indicating that a risk is beginning to materialize.

Term	Definition in the Context of This Project
Descoping	Removing lower-priority ('Could' or 'Should') features in a pre-agreed order to protect core 'Must-have' deliverables and schedule quality.
MoSCoW Prioritization	Requirement prioritization framework categorizing items into Must have, Should have, Could have, and Won't have.
IDOR (Insecure Direct Object Reference)	An access-control vulnerability where an authenticated user manipulates an account or transaction identifier to access another customer's data (prevented by server-side ownership checks returning HTTP 404).
Architecture Spike	A focused, time-boxed technical experiment conducted in Week 1 to validate SQLite WAL concurrency and BIGINT piaster round-trip precision.
WAL (Write-Ahead Logging)	An SQLite journal mode that improves concurrency and transactional integrity by appending changes to a write-ahead log before committing to the main database file.
Piaster Storage (BIGINT)	Storing monetary amounts as exact integer piasters (1 EGP = 100 piasters) in SQLite to eliminate floating-point rounding errors.